Training models

Gradient descent, and the models it fits

LECTURE 4 | GÉRON CH. 4

How a model is *fitted*, when there is no formula for the answer

---

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Worked example: the Titanic passenger manifest

Where we are

- Lecture 2 derived the **normal equation** $\hat{\boldsymbol{\theta}} = (\mathbf{X}^\intercal \mathbf{X})^{-1} \mathbf{X}^\intercal y$ — a closed form for the least-squares fit, valid exactly when $\mathbf{X}$ has full column rank
- Lecture 3 measured a classifier properly: the confusion matrix, precision and recall, and an operating point chosen against a stated constraint

Both lectures called `.fit()` and moved on.

Today we open it.

Why the closed form runs out

Situation	What the normal equation does
Many features	inverts an $(n+1) \times (n+1)$ matrix — cost grows like n^3
Many instances	forms $\mathbf{X}^\intercal \mathbf{X}$, so every row must be seen at once
Rank-deficient design	the inverse does not exist
✗ **A cost that is not squared error**	✗ **there is no closed form at all**

The last row is the one that matters. The classifier we build today minimises a cost whose stationary equations have no solution in elementary functions — so the answer has to be *walked to*.

Today

min	What
0–10	where we are, and why a formula is not always available
10–30	**the mathematics** — gradient descent, derived
30–70	**the method** — linear regression by descent, polynomial features, logistic regression on the Titanic manifest
70–85	softmax regression, decision boundaries, which solver to use
85–90	the notebook

One dataset runs through the second half: 891 passengers, and the question *how likely is this person to survive* — a probability, not a label.

The derivation in full, and logistic and softmax regression: the model, the log-loss, coefficients as log-odds, and the two conditions under which the fit is not well defined. Which solver is fastest, wall-clock

EXAMINABLE

NOT EXAMINABLE

THE MATHEMATICS

Gradient descent

Géron §4.2 — the algorithm that fits almost every model in the second half of this course

20 minutes · examinable

The question

You have a cost $J(\boldsymbol{\theta})$ and no formula for its minimiser.

How do you find one, and how do you know you have?

> **THREE THINGS THE ANSWER MUST SUPPLY**
>
> A **direction** to move in, a **step size** that does not overshoot, and a **reason** to believe the point you stop at is the minimum and not merely a place where you stopped.

The setting, fixed for the whole derivation

- m instances, n features; $\mathbf{X}$ is $m \times (n+1)$ with a leading column of ones
- $\boldsymbol{\theta} \in \mathbb{R}^{n+1}$ — the parameters, including the bias θ_0
- $\eta > 0$ — the *learning rate*, a step size

$$J(\boldsymbol{\theta}) \;=\; \mathrm{MSE}(\boldsymbol{\theta}) \;=\; \frac{1}{m}\,\|\mathbf{X}\boldsymbol{\theta} - y\|_2^2 \;=\; \frac{1}{m}\sum_{i=1}^{m}\left(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i}\right.$$

We derive on squared error because it is the cost we already know the answer for — which makes it the one cost where the method can be *checked* against a formula.

Step 1 · The cost is a surface

J maps each of the $n+1$ parameters to one number. Fitting is finding the lowest point of that surface.

> **THE ALGORITHM, IN ONE SENTENCE**
>
> Start anywhere. Measure which way is downhill. Take a step. Repeat until the downhill direction is flat.

Everything difficult is inside *which way is downhill* and *how big a step*. Those are the next two steps.

Step 2 · Which way is downhill

Move a small distance in a unit direction $\mathbf{u}$. To first order,

$$J(\boldsymbol{\theta} + \epsilon\,\mathbf{u}) - J(\boldsymbol{\theta}) \;=\; \epsilon\,\nabla J(\boldsymbol{\theta}) \cdot \mathbf{u} \;+\; o(\epsilon)$$

By Cauchy–Schwarz, $|\nabla J \cdot \mathbf{u}| \le \|\nabla J\|$ for every unit $\mathbf{u}$, with equality only when $\mathbf{u} \parallel \nabla J$:

$$\mathbf{u}^{*} \;=\; -\,\frac{\nabla J(\boldsymbol{\theta})}{\|\nabla J(\boldsymbol{\theta})\|}$$

This is *why* the gradient and not some other vector: it is the unique direction of steepest decrease, and the argument is three lines long.

Step 3 · One partial derivative

$$
\begin{aligned}
\frac{\partial J}{\partial \theta_j} &= \frac{1}{m} \sum_{i=1}^{m} 2\left(\boldsymbol{\theta}^\mathsf{T}\mathbf{x}^{(i)} - y^{(i)}\right) \cdot \frac{\partial}{\partial \theta_j}\left(\boldsymbol{\theta}^\mathsf{T}\mathbf{x}^{(i)}\right) \\
&= \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^\mathsf{T}\mathbf{x}^{(i)} - y^{(i)}\right) x_j^{(i)}
\end{aligned}
$$

Chain rule on the square, then $\partial(\boldsymbol{\theta}^\mathsf{T}\mathbf{x})/\partial\theta_j = x_j$ because the sum $\sum_k \theta_k x_k$ contains θ_j exactly once.

Read it as: **error × feature**, averaged over the instances.

Step 4 · Stack the partials

$$\nabla J(\boldsymbol{\theta}) \;=\; \frac{2}{m}\,\mathbf{X}^{\mathsf{T}}\big(\mathbf{X}\boldsymbol{\theta} - y\big)$$

THE GRADIENT OF THE MEAN SQUARED ERROR

The j-th entry of $\mathbf{X}^{\mathsf{T}}\mathbf{r}$ is $\sum_i r_i x_j^{(i)}$, which is exactly the sum in step 3 with $\mathbf{r} = \mathbf{X}\boldsymbol{\theta} - y$ the residual vector.

✓ One matrix product over the whole training set. Cost per evaluation: $O(mn)$ — linear in the rows, where the normal equation was cubic in the columns.

Step 5 · The update rule

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \eta \nabla J\left(\boldsymbol{\theta}^{(k)}\right) = \boldsymbol{\theta}^{(k)} - \frac{2\eta}{m} \mathbf{X}^{\mathsf{T}}\left(\mathbf{X}\boldsymbol{\theta}^{(k)} - y\right)$$

BATCH GRADIENT DESCENT

Batch, because every step uses the whole training set. Three lines of NumPy, and no matrix is ever inverted.

η has units: it converts a gradient into a displacement in parameter space. That is why its right value depends on how the features are scaled — step 8.

Step 6 · Where it stops

The iteration stops moving exactly when the step is zero:

$$\nabla J(\hat{\boldsymbol{\theta}}) = \mathbf{0} \iff \mathbf{X}^\intercal\left(\mathbf{X}\hat{\boldsymbol{\theta}} - y\right) = \mathbf{0} \iff \mathbf{X}^\intercal\mathbf{X}\hat{\boldsymbol{\theta}} = \mathbf{X}^\intercal y$$

✓ That is the normal equation of Lecture 2.

Descent does not solve a different problem. It solves the same system, without ever forming $(\mathbf{X}^\intercal\mathbf{X})^{-1}$ — which is why it survives the rank-deficient case that the closed form cannot even state.

Step 7 · Is that point the minimum?

The Hessian of the MSE is constant:

$$\nabla^2 J = \frac{2}{m}\mathbf{X}^\top\mathbf{X}, \qquad \mathbf{v}^\top\mathbf{X}^\top\mathbf{X}\mathbf{v} = \|\mathbf{X}\mathbf{v}\|_2^2 \geq 0$$

Positive semi-definite everywhere, so J is **convex**: it has no local minima, no plateaux other than the minimum itself, and no saddle points.

✓ On a convex cost, a stationary point *is* a global minimum. Descent cannot get stuck — it can only be slow.

Hold on to the word *convex*. The moment we leave linear models, in Lecture 9, every one of those guarantees goes.

Step 8 · How large may η be?

Write $H = \nabla^2 J$ and subtract the fixed point from the update:

$$\boldsymbol{\theta}^{(k+1)} - \hat{\boldsymbol{\theta}} = \left(\mathbf{I} - \eta H\right)\left(\boldsymbol{\theta}^{(k)} - \hat{\boldsymbol{\theta}}\right)$$

In the eigenbasis of H each coordinate of the error is multiplied by $(1 - \eta\lambda_i)$ every step, so the error decays for all of them if and only if

$$0 < \eta < \frac{2}{\lambda_{\max}}$$

Because J is quadratic the argument is exact, not a local approximation.

What each failure looks like

Learning rate	Factor per step	What you see
η far below $2/\lambda_{\max}$	$1 - \eta\lambda_i$ just under 1	the cost falls, slowly, and is still falling when you stop
✓ $\eta \approx 1/\lambda_{\max}$	✓ **small**	✓ **the cost drops fast and flattens**
✗ $\eta > 2/\lambda_{\max}$	✗ $\lvert 1 - \eta\lambda_i \rvert > 1$	✗ **the cost *rises*, oscillates, and overflows to** `nan`

FAILURE CONDITION — DIVERGENCE IS SILENT UNTIL IT IS LOUD

A diverging fit does not raise. It returns weights full of `inf` and a model that predicts one value everywhere. **Plot the cost against the step number**; it is the only cheap way to see which of the three rows you are in.

Step 9 · Why you must scale the features

The step count is governed by the slowest mode. With the best fixed η, the error contracts per step by a factor

$$\frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \text{the condition number of } \mathbf{X}^\intercal \mathbf{X}$$

- $\kappa = 1$ — circular bowl, one step
- $\kappa = 100$ — a long thin valley; the path zig-zags across it
- κ large because one column is in thousands and another in units — a scaling accident, and entirely avoidable

✓ Standardising the columns is not cosmetic. It changes κ, and κ is what sets the number of steps.

Step 10 · One instance at a time

Each batch step costs $O(mn)$, which for large m is unaffordable. Draw an index i uniformly at random and use only that row:

$$\mathbf{g}^{(i)}(\boldsymbol{\theta}) \;=\; 2\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)\,\mathbf{x}^{(i)}$$

Take the expectation over the draw:

$$\mathbb{E}_i\left[\mathbf{g}^{(i)}\right] \;=\; \sum_{i=1}^{m} \frac{1}{m}\, 2\big(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)} - y^{(i)}\big)\mathbf{x}^{(i)} \;=\; \nabla J(\boldsymbol{\theta})$$

✓ The one-instance gradient is an unbiased estimate of the batch gradient.

Step 11 · Unbiased is not the same as right

The estimate's variance does *not* shrink as $\boldsymbol{\theta}$ approaches $\hat{\boldsymbol{\theta}}$: at the minimum the batch gradient is zero and the individual gradients are not. So the iterates bounce around the minimum forever, and the remedy is to shrink the step — a **learning schedule** $\eta_k \to 0$. Averaging over b rows instead of 1 is still unbiased, by the same one-line argument, and divides the variance by b.

Variant	Rows per step	Cost per step	Path
Batch	m	$O(mn)$	smooth, straight to the minimum
Mini-batch	b	$O(bn)$	slightly noisy, and vectorises on hardware
Stochastic	1	$O(n)$	very noisy, never settles without a schedule

The bouncing is not only a cost. On a non-convex surface — every network from Lecture 9 onwards — it is what lets the iterate leave a bad local minimum. Here, on a convex bowl, it is pure nuisance.

The derivation, in one page — 1–5

1. 1 $J = \frac{1}{m}\|\mathbf{X}\boldsymbol{\theta} - y\|^2$
   the cost, as a surface over parameter space

2. 2 $-\nabla J$ is the steepest-descent direction
   Cauchy–Schwarz on the directional derivative

3. 3 $\partial J / \partial \theta_j = \frac{2}{m}\sum_i (\boldsymbol{\theta}^\mathsf{T}\mathbf{x}^{(i)} - y^{(i)})\, x_j^{(i)}$
   chain rule; error times feature

4. 4 $\nabla J = \frac{2}{m}\mathbf{X}^\mathsf{T}(\mathbf{X}\boldsymbol{\theta} - y)$
   the partials, stacked into one matrix product

5. 5 $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\nabla J$
   the update; η turns a gradient into a displacement

The derivation, in one page — 6–10

1. 1 $\nabla J = \mathbf{0} \iff \mathbf{X}^\intercal \mathbf{X}\boldsymbol{\theta} = \mathbf{X}^\intercal y$
   the fixed point is the normal equation

2. 2 $\nabla^2 J = \frac{2}{m}\mathbf{X}^\intercal \mathbf{X} \succeq 0$
   convex, so that stationary point is the global minimum

3. 3 $0 < \eta < 2/\lambda_{\max}$
   every eigen-mode of the error must contract

4. 4 contraction $(\kappa - 1)/(\kappa + 1), \kappa = \lambda_{\max}/\lambda_{\min}$
   conditioning sets the step count — so scale the columns

5. 5 $\mathbb{E}_i[\mathbf{g}^{(i)}] = \nabla J$
   one row is an unbiased gradient — what makes mini-batch descent legitimate

What the derivation buys you

The result	What it lets you do today
$\nabla J = \frac{2}{m}\mathbf{X}^\intercal(\mathbf{X}\boldsymbol{\theta} - y)$	fit a linear model without inverting anything
Convexity	trust the answer whatever the starting point — and know that this stops being true in Lecture 9
$\eta < 2/\lambda_{\max}$	read a rising cost curve as a step-size fault, not a bug
Contraction $(\kappa - 1)/(\kappa + 1)$	know that scaling is part of the algorithm, not tidiness
✓ Unbiasedness of $\mathbf{g}^{(i)}$	**✓ use mini-batches on data that does not fit in memory**

✓ And one more: the derivation never used the fact that the cost was squared error until step 3. Swap the cost, redo step 3, and everything else stands. That is what we do in twenty minutes.

THE METHOD

Fitting by descent

40 minutes · examinable

Two routes to the same $\hat{\theta}$

	Normal equation / SVD	Gradient descent
Cost in m	linear	linear, per step
Cost in n	✗ $n^{2.4}$ to n^3	**linear ✓**
Out-of-core	**✗ no — needs all rows**	**yes, with mini-batches ✓**
Scaling required	no	**✗ yes**
Hyperparameters	none	**✗ η, batch size, schedule, stopping rule**
Other costs	**✗ squared error only**	**any differentiable cost ✓**

When to prefer each. Few features and data that fits in memory: use the closed form, it has no knobs to get wrong. Many features, streaming data, or any cost that is not squared error: descent.

The batch step, written out

```
theta = rng.standard_normal((n + 1, 1))        # start anywhere

for step in range(n_steps):
    grad  = 2 / m * X_b.T @ (X_b @ theta - y)  # the gradient of step 4
    theta = theta - eta * grad                 # the update of step 5
```

Two lines. No inverse, no decomposition, no solver. In practice you call `SGDRegressor`, which is this loop plus a schedule, a stopping rule and a penalty — inside a pipeline, so the scaler is fitted per fold: `make_pipeline(StandardScaler(), SGDRegressor(eta0=0.01))`.

FAILURE CONDITION — UNSCALED FEATURES

One column in thousands and another in units makes κ enormous. The cost falls quickly for a hundred steps and then crawls; raise η to compensate and the large-eigenvalue direction diverges. **One cause, two symptoms, and neither is fixed by more iterations.**

Curvature without leaving linearity

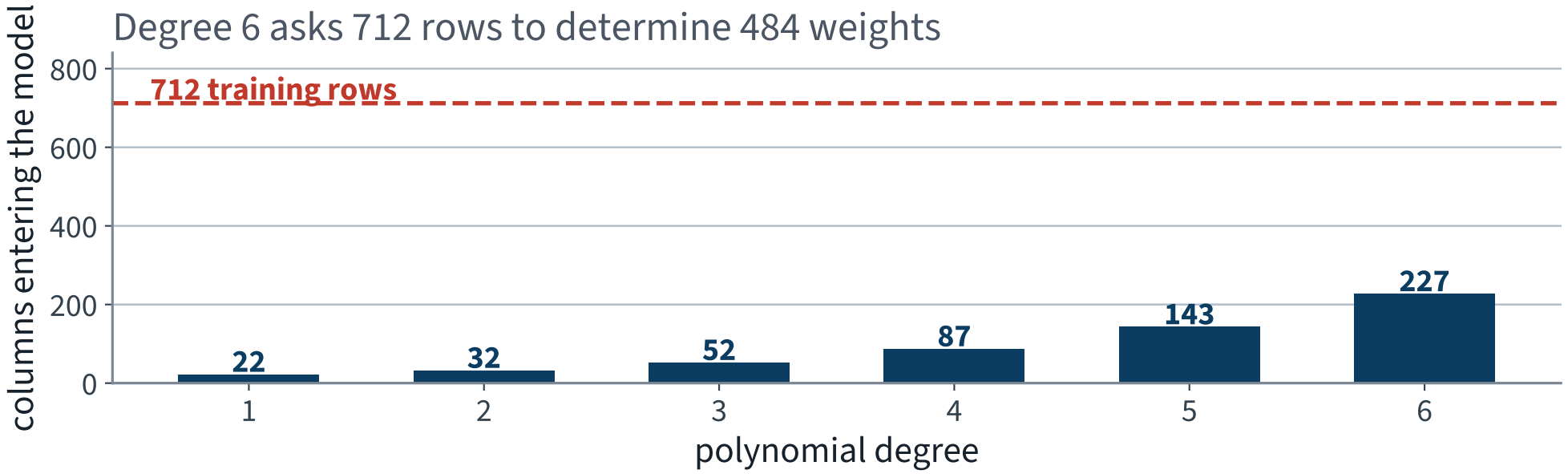
A straight line cannot fit a curve. Add the powers as *columns*:

$$\hat{y} \;=\; \theta_0 + \theta_1 x + \theta_2 x^2 \quad \text{is linear in } \boldsymbol{\theta},\ \text{not in } x$$

Everything derived above still applies unchanged — the gradient, the convexity, the convergence condition — because all of them were statements about $\boldsymbol{\theta}$.

`PolynomialFeatures(degree=d)` also generates every *cross* product, which is how a linear model represents an interaction such as "third class *and* travelling alone". The count is $\binom{n+d}{d}$ — the number of monomials of degree at most d — so "a bit more capacity" is never a small change.

What that costs in columns



Degree 6 on four numeric columns gives the model 227 weights to determine from 712 passengers — about three rows per weight.

THE WORKED EXAMPLE

A probability, not a label

891 passengers, and a question that rules out most of the models in this book

The task

> **MARINE SAFETY PLANNING UNIT**
>
> “We are building an evacuation-assistance model from historical passenger manifests. We need to know, for each passenger profile, **how likely that person is to get off unaided** — and we need to be able to explain the assignment afterwards.”

What they asked for	What that forces
“how likely”	a **probability**, not a label
the number is used to allocate	it must be **calibrated**
“explain the assignment”	weights a human can **read**

✓ “This passenger survives” is not checkable on its own. “This passenger survives with probability 0.31” is: take everyone the model puts near 0.31 and count. That is why requirement 1 is not a formatting preference.

The data

```
def load_titanic():
    tarball = Path("datasets/titanic.tgz")
    if not tarball.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/titanic.tgz"
        urllib.request.urlretrieve(url, tarball)
        with tarfile.open(tarball) as t:
            t.extractall(path="datasets", filter="data")
    return pd.read_csv("datasets/titanic/train.csv")
```

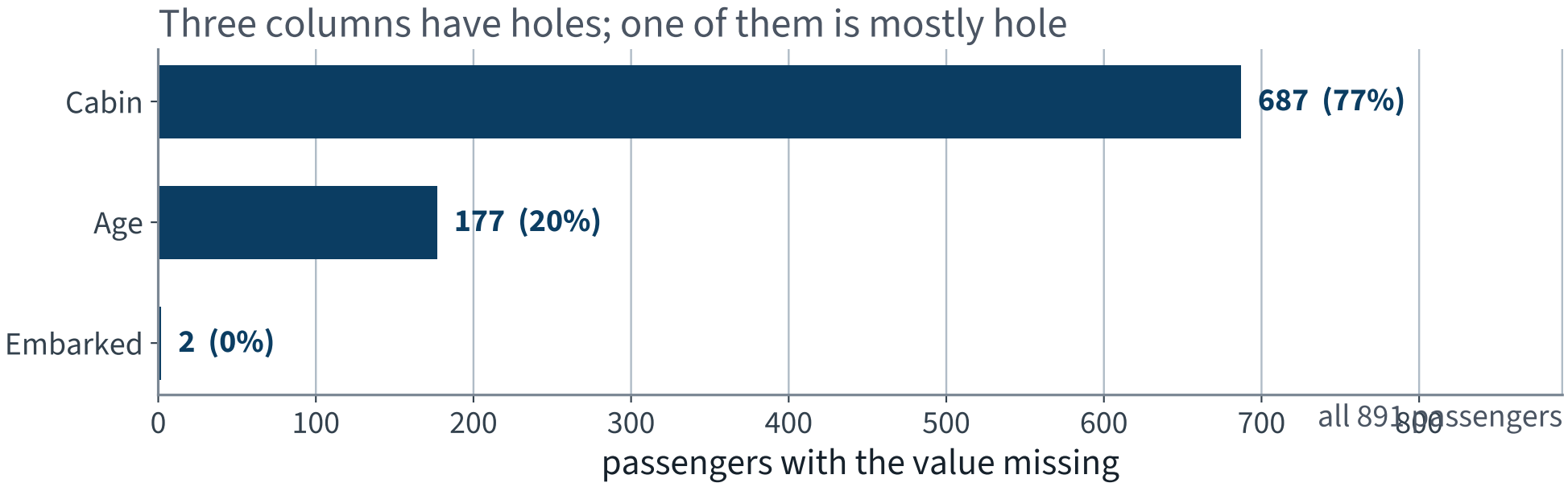
891 passengers, 12 columns — the whole labelled corpus. The archive also holds `test.csv`, which has **no `Survived` column**: it is a competition submission file, and it is useless for measuring anything. We hold out our own test set.

What is in the twelve columns

Column	Meaning	Column	Meaning
`Survived`	the label, 0 or 1	`Parch`	parents and children aboard
`Pclass`	ticket class, 1–3	`Ticket`	ticket number, free text
`Name`	full name, free text	`Fare`	fare paid
`Sex`	male or female	`Cabin`	cabin number, mostly absent
`Age`	years, sometimes absent	`Embarked`	port of embarkation
`SibSp`	siblings and spouses aboard	`PassengerId`	a row number

✗ **`PassengerId` is an index dressed as a feature. Give it to a model and the model will find something in it.**

Three columns have holes in them



Cabin is missing for 77% of the passengers. That is not a column with gaps; it is a gap with a column.

Three holes, three different answers

- `Cabin`, 687 missing — do not impute it. Take the deck letter where there is one and give the rest their own level `U`: the *missingness* is the information
- `Age`, 177 missing — impute the median *inside the pipeline*, so it is recomputed on every training fold
- `Embarked`, 2 missing — impute the most frequent port. Two rows out of 891 cannot matter, and you should still be able to say what you did

✗ `Deck = "U"` will largely mean *third class*, because the cabin number was recorded for the passengers whose tickets carried one. So the column is partly a proxy for `Pclass`. Write that down.

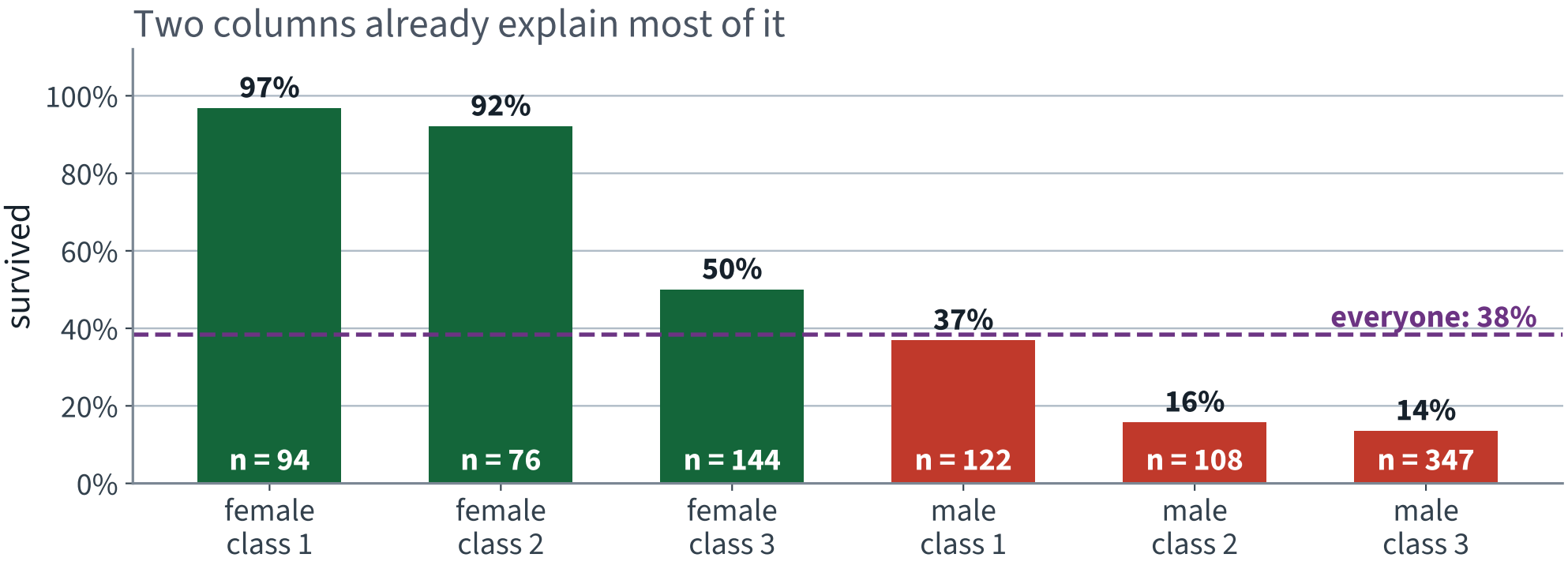
The label

Outcome	Passengers	Share
Did not survive	549	61.6%
Survived	342	38.4%

Mildly imbalanced — nothing like the rare-event problem of the previous lecture, where the positive class was one row in ten. Accuracy will not be absurd here.

✗ It will be something more dangerous than absurd: plausible.

Two columns already explain most of it



A first-class woman and a third-class man differ by a factor of seven. Any model that cannot beat this table is not earning its keep.

`Name` is not a useless string

```
Braund, Mr. Owen Harris
Cumings, Mrs. John Bradley (Florence Briggs Thayer)
Heikkinen, Miss. Laina
```

Every name carries a **title**, and the title carries sex, marital status and — for `Master` — being a boy.

Title	Passengers	Survived
`Mrs`	126	79.4%
`Miss`	185	70.3%
`Master` — a boy	40	57.5%
`Rare` — Dr, Rev, Col, …	23	34.8%
`Mr`	517	15.7%

`Master` is the only level that is not a re-encoding of `Sex`: those 40 boys survived at nearly four times the rate of the

Four engineered columns, each with a reason

```
d["FamilySize"] = d["SibSp"] + d["Parch"] + 1
d["IsAlone"]    = (d["FamilySize"] == 1).astype(int)
d["Deck"]       = d["Cabin"].str[0].fillna("U")
# Title, extracted from Name as above
```

Column	The claim it encodes
`Title`	a boy is not a man
`FamilySize`	a group evacuates together, and slowly
`IsAlone`	nobody is waiting for you
`Deck`	distance to a boat, and a proxy for class

✗ The first line is an exact linear function of two columns we already have. Remember it — we come back to it with a rank computation, not with an opinion.

Split first — still

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

assert len(X_train) == 712 and len(X_test) == 179
print(f"train rate {y_train.mean():.4f}   test rate {y_test.mean():.4f}")
# train rate 0.3834   test rate 0.3855
```

Stratified on the **label** this time, not on a predictor: with 179 test rows an unstratified draw can shift the base rate by several points, and every downstream number moves with it.

✓ 712 training passengers. That is a small dataset, and by the end of the lecture the size is doing visible damage.

Before the model: what does *nothing* score?

```
p = y_train.mean()                                    # 0.3834

majority_accuracy = 1 - p                             # 0.617
constant_log_loss = -(p*np.log(p) + (1-p)*np.log(1-p))
```

0.666

The log loss of a model that has learned **nothing except the base rate**. It is the entropy of the label, in nats, and every number below it has to be compared against it.

Logistic regression: the model

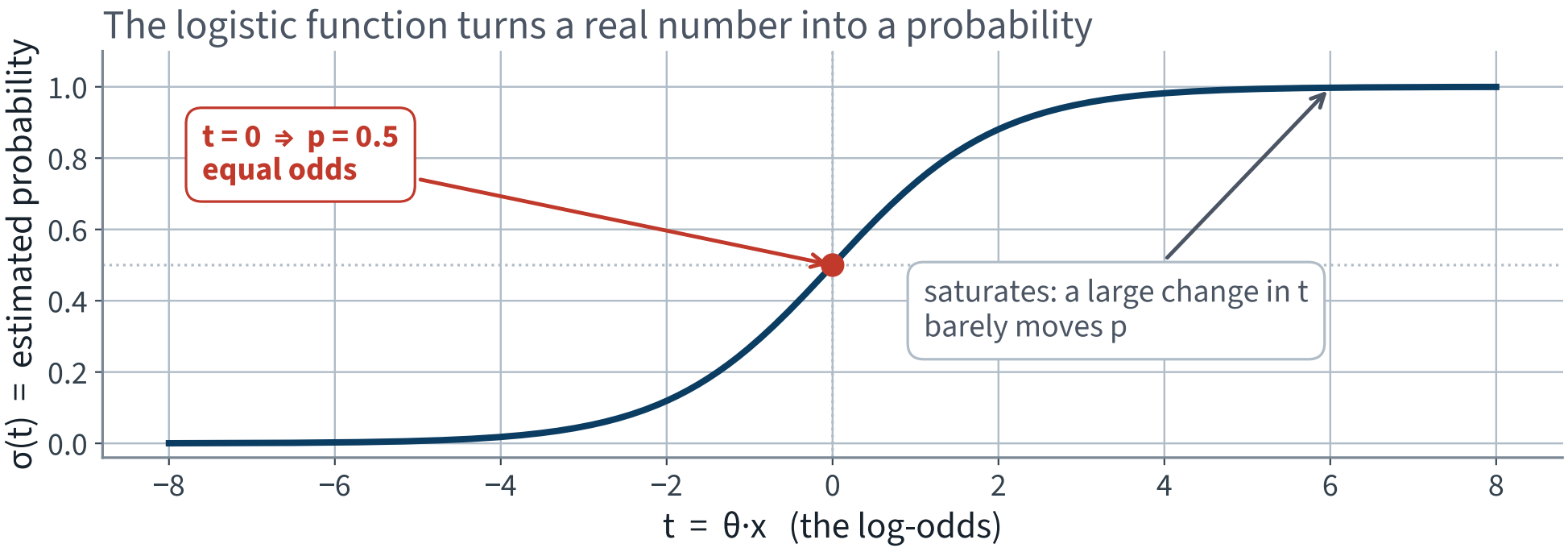
Take the linear model of Lecture 2 and pass it through one function.

$$\hat{p} = \sigma(\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}), \qquad \sigma(t) = \frac{1}{1+e^{-t}}$$

$\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$ is exactly the quantity least squares fitted. Everything new is in σ and in what we ask $\boldsymbol{\theta}$ to minimise.

σ is strictly increasing and maps $\mathbb{R}$ onto $(0, 1)$ — so it never returns exactly 0 or exactly 1, which matters when the loss takes a logarithm of it.

One function, and the whole difference



Monotone, so the ranking is the linear model's ranking. Saturating, so beyond about $|t| = 5$ more evidence changes almost nothing.

What $\boldsymbol{\theta}^\top \mathbf{x}$ *is*

Invert σ:

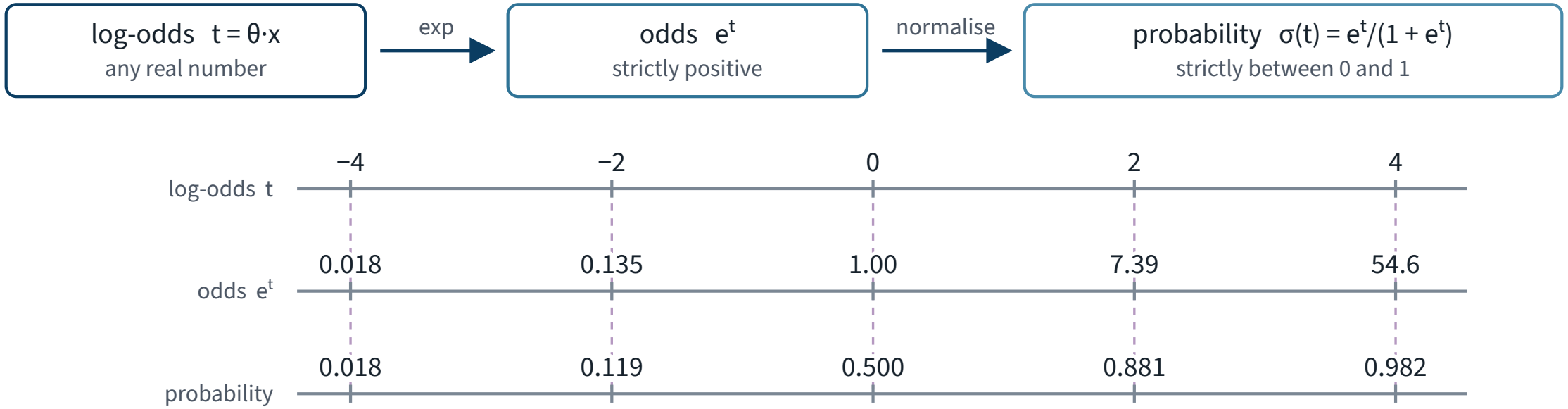
$$t = \log \frac{\hat{p}}{1-\hat{p}} = \boldsymbol{\theta}^\top \mathbf{x}$$

The linear model does not predict the probability. It predicts the **log-odds**, and the coefficients act on the odds.

✓ Add 1 to x_j and the odds are multiplied by e^{θ_j}, wherever you started. The probability has no such rule — which is the sentence most often got wrong in the examination.

Three scales, one number

One number, read on three scales



Equal steps on the top scale become equal **multipliers** on the middle one — and neither on the bottom one.
That is why a coefficient is a multiplier on the **odds**, never on the probability.

Equal steps in log-odds are equal *multipliers* in odds, and neither in probability.

What it minimises

$$J(\boldsymbol{\theta}) \;=\; -\frac{1}{m}\sum_{i=1}^{m}\Big[\,y^{(i)}\log\hat{p}^{(i)} + \big(1-y^{(i)}\big)\log\big(1-\hat{p}^{(i)}\big)\Big]$$

THE LOG LOSS, IN NATS

- Only the probability you gave the *true* class enters each term
- Unbounded above: claim 0.001 for a passenger who survived and that term is $-\log 0.001 \approx 6.9$
- It is the negative log-likelihood of the labels under the model, divided by m

Why *this* loss and not something friendlier

A scoring rule is **proper** if the expected score is optimised by reporting your true belief.

- **Log loss** — proper. Reporting anything other than the true p is expected to score worse
- **Brier score**, $\frac{1}{m}\sum(\hat{p}-y)^2$ — also proper, bounded, easier to read; we report it too

> **FAILURE CONDITION — ACCURACY IS NOT PROPER**
>
> Under accuracy a model reporting 0.51 scores exactly the same as one reporting 0.99. So accuracy gives a model *no reason at all* to tell the truth about its uncertainty, and optimising it destroys calibration.

Redo step 3 for this cost

1. [1] $\sigma'(t) = \sigma(t)\big(1 - \sigma(t)\big) = \hat{p}\,(1 - \hat{p})$
   differentiate $(1 + e^{-t})^{-1}$ and rewrite

2. [2] $\partial \ell / \partial \hat{p} = (\hat{p} - y) \,/\, \hat{p}(1 - \hat{p})$
   put the two log terms over one denominator

3. [3] $\partial J / \partial \theta_j = \frac{1}{m} \sum_i (\hat{p}^{(i)} - y^{(i)})\, x_j^{(i)}$
   chain rule — the two $\hat{p}(1 - \hat{p})$ factors cancel exactly

✓ Error times feature. The same shape as the gradient of the MSE.

and yet: no closed form

Setting that gradient to zero gives $\mathbf{X}^\top\big(\sigma(\mathbf{X}\boldsymbol{\theta}) - y\big) = \mathbf{0}$, in which $\boldsymbol{\theta}$ appears inside a transcendental function. There is no rearrangement that isolates it.

> **BUT J IS CONVEX**
>
> Its Hessian is $\frac{1}{m}\mathbf{X}^\top\mathbf{S}\mathbf{X}$ with $\mathbf{S} = \operatorname{diag}\big(\hat{p}^{(i)}(1-\hat{p}^{(i)})\big) \succeq 0$, so the surface is a bowl and gradient descent reaches the global minimum.

✓ This is the first model in the course that has no formula for its own answer. Everything in the first half of today is what makes it fittable anyway.

The preprocessing

```
NUM = ["Age", "Fare", "FamilySize", "SibSp", "Parch"]
CAT = ["Pclass", "Sex", "Embarked", "Title", "Deck"]
BIN = ["IsAlone"]

prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"),
                          StandardScaler()), NUM),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                          OneHotEncoder(handle_unknown="ignore")), CAT),
    ("bin", "passthrough", BIN),
])
```

Standardising is step 9 of the derivation, applied: the solver is doing descent, and κ decides how long it takes. Everything is inside the pipeline, so nothing is fitted on a held-out fold.

Fit it, and measure it honestly

```
clf = LogisticRegression(C=1e6, max_iter=4000)   # C=1e6 turns the penalty OFF
model = Pipeline([("prep", prep), ("clf", clf)])

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_validate(model, X_train, y_train, cv=cv,
                        scoring=["neg_log_loss", "accuracy", "neg_brier_score"],
                        return_train_score=True)
```

`StratifiedKFold`, not `KFold`: with 71 passengers per fold, an unstratified split can hand a fold a base rate several points from the truth.

✗ Scikit-learn regularises logistic regression by default — `C=1.0`. We switch it off so that today's coefficients are the data's and not a compromise with a penalty. Turning it back on is the next lecture.

The first numbers

Model	Log loss	Accuracy
Report the base rate to everyone	0.666	61.7%
Logistic regression, 10-fold cross-validated	**0.496 ✓**	**81.5% ✓**

About a quarter below the anchor on log loss, and twenty points of accuracy above the majority class.

Cross-validated, not training. And the per-fold spread of the log loss is **0.22** — larger than every difference we are about to compare. Carry that number; it is why the next two slides ask a question other than *which is better*.

Read the coefficients

```
names = model[:-1].get_feature_names_out()
coefs = model[-1].coef_[0]
for n, c in sorted(zip(names, coefs), key=lambda t: -abs(t[1]))[:6]:
    print(f"{n.split('__')[-1]:16s} {c:+.3f}   odds x{np.exp(c):.3f}")
```

```
Deck_T           -5.678   odds x0.003
Title_Master     +3.484   odds x32.596
Sex_female       +3.308   odds x27.325
Sex_male         -2.833   odds x0.059
Title_Miss       -2.806   odds x0.060
Title_Mrs        -2.020   odds x0.133
```

Three rows that cannot be right

- `Title_Miss` at −2.81 — but 70.3% of them survived, against 38.4% overall
- `Title_Mrs` at −2.02 — the highest-surviving group in the data
- `Deck_T` is the largest weight in the model — and there is exactly **one** passenger on deck T

✗ These are not weak effects or noisy estimates. They have the wrong sign, and the model's predictions are nonetheless good.

Before reaching for a story about confounding, ask the cheaper question: *what is the shape of the design matrix?*

Count the columns, then count the rank

```
Z   = model[:-1].transform(X_train)
Z_b = np.c_[np.ones(len(Z)), Z]          # add the intercept column

print("columns:", Z_b.shape[1])          # 29
print("rank:   ", np.linalg.matrix_rank(Z_b))   # 23
print("cond:   ", np.linalg.cond(Z_b))          # 2.6e+16
```

✗ Six columns of the design matrix are linear combinations of the others.

Where the six come from

Five from the encoder. Each one-hot block sums to 1 in every row — and the intercept column is also 1 in every row. Five categorical columns, five exact dependencies.

One from us.

`FamilySize = SibSp + Parch + 1`, exactly, for every passenger.

```
resid = (X_train["SibSp"] + X_train["Parch"] + 1
         - X_train["FamilySize"]).abs().max()
print(resid)          # 0.0
```

Not “highly correlated” — an identity. Standardising does not help: an affine map of a linear dependence is still a linear dependence.

What a rank-deficient design does to the fit

$$\mathbf{X}\mathbf{v} = \mathbf{0}, \quad \mathbf{v} \neq \mathbf{0}$$

$$\Longrightarrow \quad \mathbf{X}\left(\boldsymbol{\theta} + c\,\mathbf{v}\right) = \mathbf{X}\boldsymbol{\theta} \quad \text{for every } c$$

Recall from Lecture 2 that $\mathbf{X}^\top\mathbf{X}$ is invertible exactly when $\mathbf{X}$ has full column rank. Logistic regression has no normal equation, but the same condition governs it: along $\mathbf{v}$ the loss is flat.

FAILURE CONDITION — THE MINIMISER IS NOT UNIQUE

There is not one best $\boldsymbol{\theta}$ but a whole affine subspace of them. The solver returns one point of it and says nothing. Predictions are unaffected; **every claim about a coefficient is void**.

Prove it on our own matrix

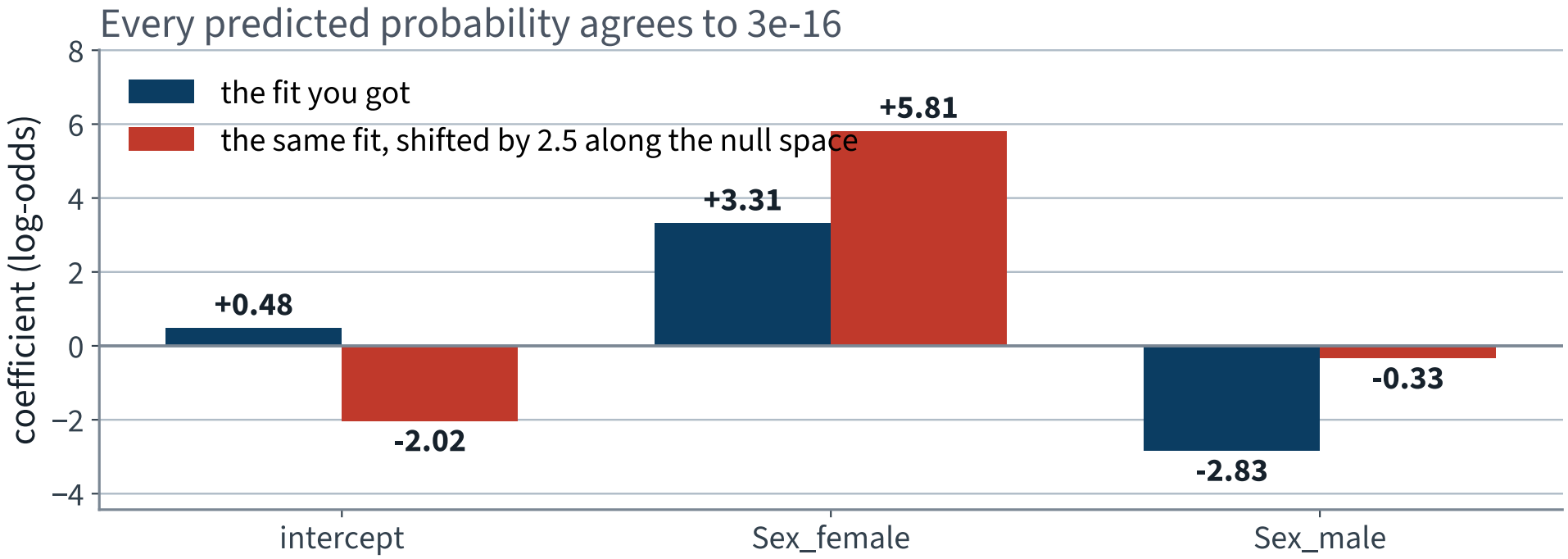
Add 2.5 to `Sex_female` and to `Sex_male`, subtract 2.5 from the intercept. Nothing else.

```
v = np.zeros_like(theta)
v[names.index("Sex_female")] = 1.0
v[names.index("Sex_male")]   = 1.0

logit_a = Z @ theta              + b0
logit_b = Z @ (theta + 2.5*v) + (b0 - 2.5)
print(np.abs(logit_a - logit_b).max())     # 1.8e-15
```

✗ `Sex_male` can be −2.83 or −0.33. Both fits make the same prediction for all 712 passengers.

Two coefficient vectors, one model



✗ If a coefficient can be moved without changing any prediction, that coefficient is not a property of the data.

The repair: delete the redundancy

```
NUM = ["Age", "Fare", "SibSp", "Parch"]        # FamilySize removed

("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
                      OneHotEncoder(drop="first", min_frequency=2,
                                    handle_unknown="infrequent_if_exist")), CAT),
```

Design matrix	Columns	Rank	Condition number
As first written	29	✗ 23	✗ 2.6e+16
One level dropped, FamilySize removed	23	23 ✓	83.6 ✓

Fourteen orders of magnitude. `IsAlone` stays: it is an indicator, not a linear function of the counts.

What `drop="first"` costs you

Each dropped level becomes the **reference**, folded into the intercept. Every remaining coefficient is read *relative to it*.

Column	Reference level	Column	Reference level
`Pclass`	1st class	`Title`	`Master`
`Sex`	`female`	`Deck`	`A`
`Embarked`	Cherbourg		

✗ A coefficient quoted with no stated reference level cannot be read by anybody, including you in a month. This is the most common defect in the regression tables you will be asked to review.

And `min_frequency` rather than `handle_unknown="ignore"`: with `drop="first"`, an ignored unknown level is encoded all-zeros, which is the encoding of the *reference* level — so our one deck-T passenger would be scored as though they were on deck A whenever a fold holds them out.

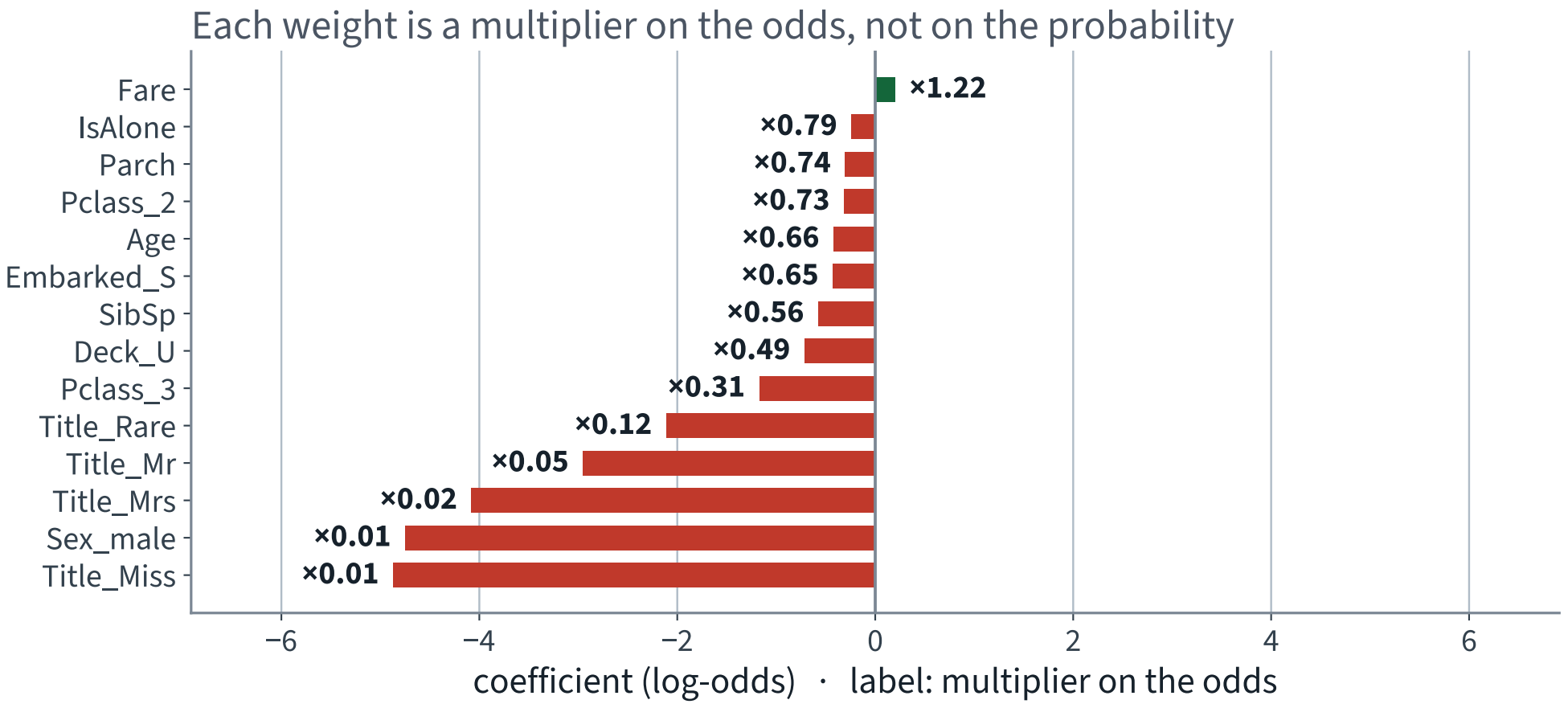
And the honest number moved

Encoding	Cross-validated log loss
All levels, FamilySize kept — rank 23 of 29	0.496
One level dropped, FamilySize removed — full rank	**0.468 ✓**

Not a large move, and not the point. The point is that the first row was produced by a fit that had no unique answer, and nothing said so.

The fold spread of the repaired model is 0.13, and of the rank-deficient one 0.22 — both far wider than this gap. Do not claim the second encoding is *better* on this evidence; claim it is *defined*.

Now the coefficients can be read



Each bar is a log-odds. The label beside it is what the odds get multiplied by.

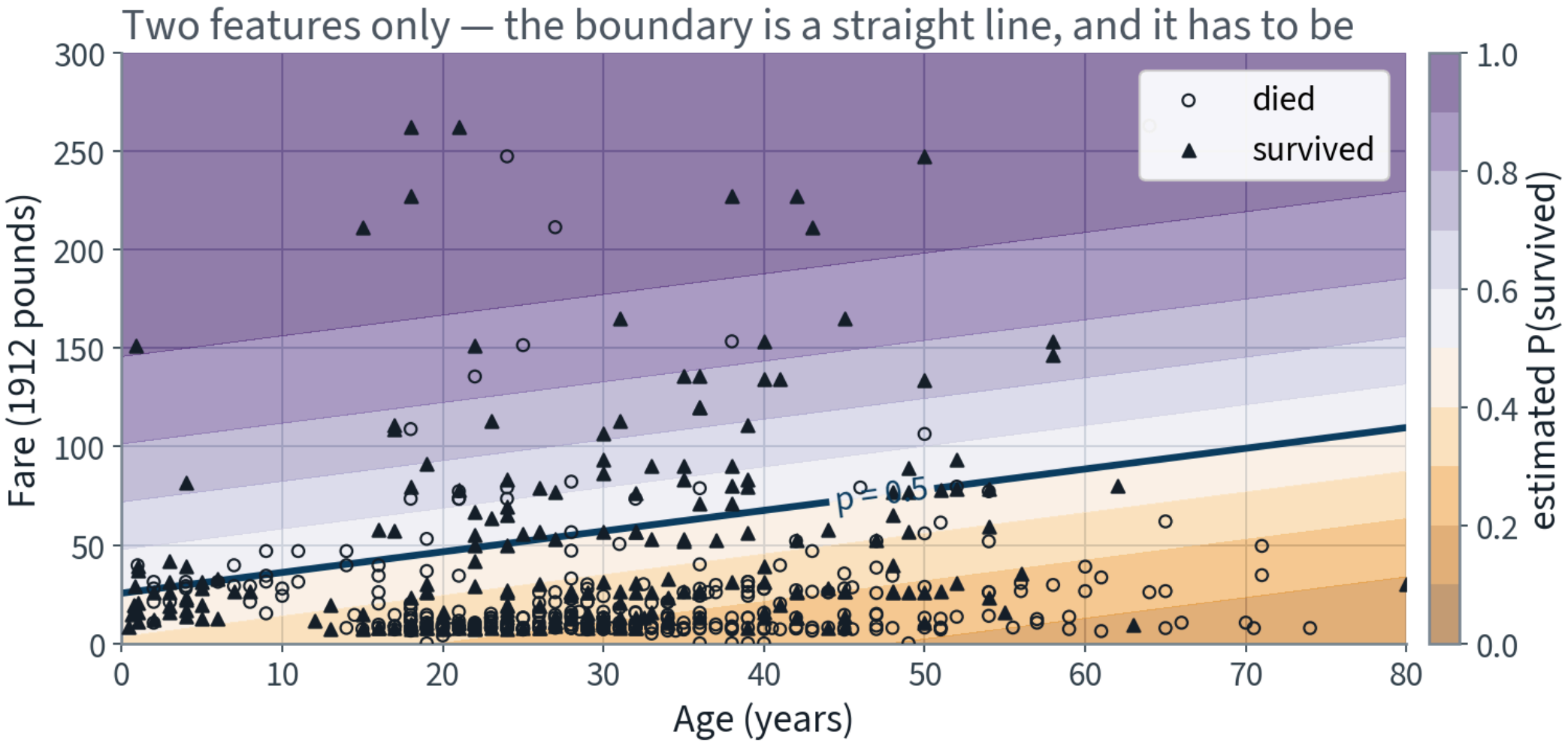
Reading one of them out loud

> `Pclass_3` = -1.169
>
> "Holding every other recorded feature fixed, travelling third class multiplies the *odds* of survival by $e^{-1.169} = 0.31$, relative to first class."

- "Holding every other feature fixed" is doing real work — drop it and the sentence is false
- Odds, not probability. Multiplying odds of 1 by 0.31 gives $p = 0.24$; doing it from $p = 0.9$ gives 0.74
- Relative to the reference level, always

✗ Unique is not the same as *stable*: `Sex` and `Title` still carry much of the same information, so how the effect is split between them moves with the sample. We only fixed uniqueness.

The decision boundary, on two features



Every contour is straight: the log-odds are linear. The sigmoid bends the *colours*, never the *lines*.

What a straight boundary cannot express

“Children and the elderly were helped first” is not a monotone statement about age. A linear term in `Age` cannot say it.

> **THE FIX IS THE ONE FROM TWENTY MINUTES AGO**
>
> Add Age^2. The log-odds become quadratic in age, the boundary in the original coordinates becomes a conic — and the model is still linear in $\boldsymbol{\theta}$, so nothing in the derivation changes.

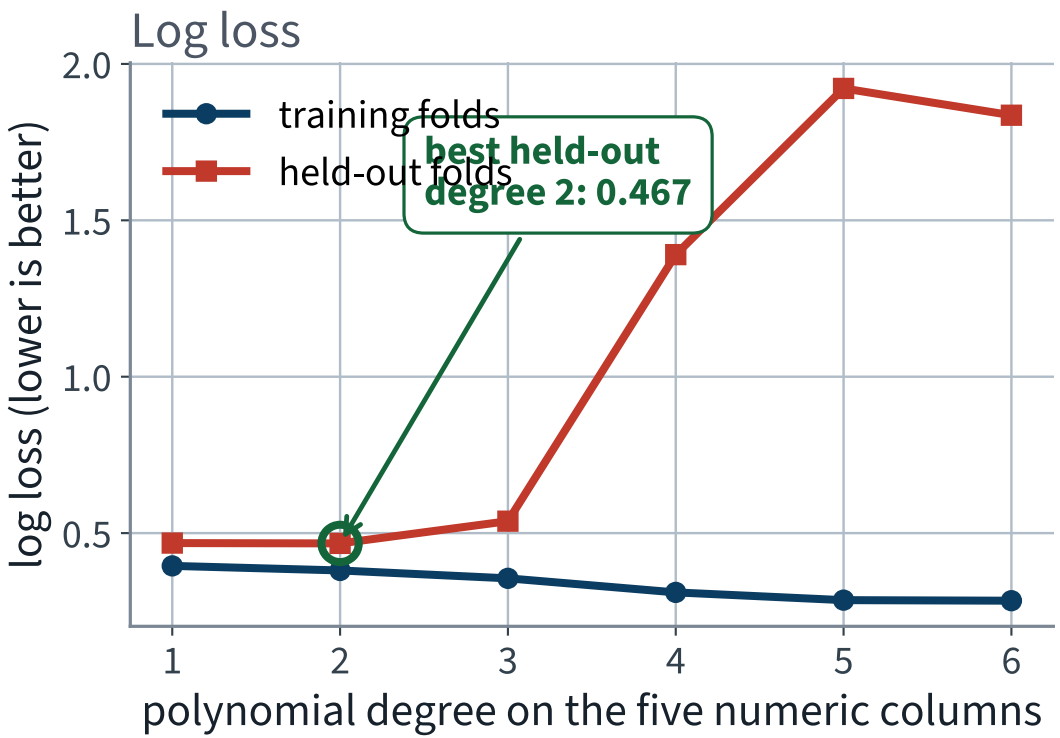
Applied to the four numeric columns only: squaring a one-hot indicator returns the indicator, and we have just spent ten minutes removing exact copies.

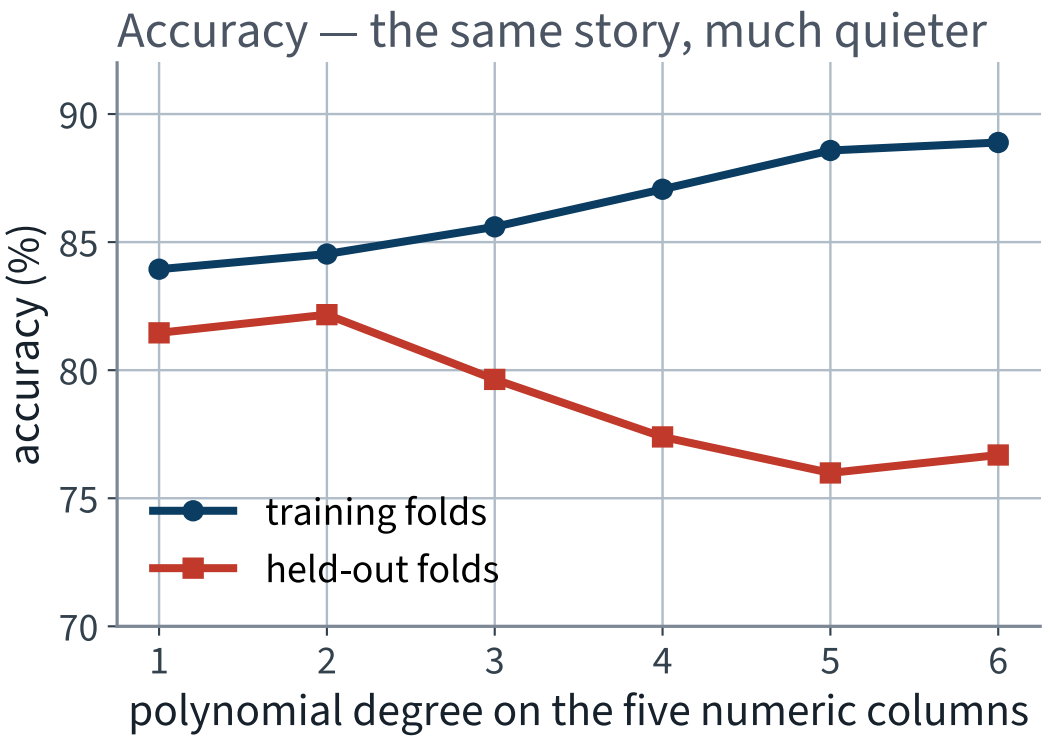
Push the capacity up, and record *both* curves

Degree	Columns	Training log loss	Held-out log loss	Held-out accuracy
1	22	0.395	0.468	81.5%
2	32	0.381	**0.467 ✓**	**82.2% ✓**
3	52	0.355	0.538	79.6%
4	87	0.310	**✗ 1.390**	77.4%
5	143	0.286	**✗ 1.922**	76.0%
6	227	0.284	**✗ 1.836**	76.7%

Degrees 1 and 2 differ by 0.001 in held-out log loss and the fold spread is over a hundred times that. **✗ Those two are a tie**; reporting degree 2 as the winner would be reporting noise.

The two curves





✗ The training curve never stops improving. The held-out curve turns at degree 2 and then falls off a cliff.

Log loss saw it first

From degree 2 to degree 5, held-out accuracy falls by about six points. Held-out log loss is multiplied by more than **four**.

- Accuracy only notices when a prediction crosses the cut-off
- Log loss notices a confident prediction becoming a confidently *wrong* one
- The failure is not that the model gets more passengers wrong — it is that it becomes *certain* about the ones it gets wrong

✓ The requirement was for probabilities. The metric that matches the requirement is the metric that saw the damage.

A warning worth reading

```
ConvergenceWarning: lbfgs failed to converge after 4000 iteration(s)
```

Degree	1	2	3	4	5	6
Converged	yes ✓	yes ✓	yes ✓	✗ no	✗ no	✗ no
Iterations used	74	256	935	4000	4000	4000

The reflex is to raise `max_iter`. It cannot help, and understanding why is the point.

Why more iterations cannot help

With 87 columns and 712 rows the two classes eventually become **linearly separable**. Suppose some $\boldsymbol{\theta}$ separates them perfectly, so $\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}^{(i)}$ has the right sign for every i. Then

$$c \to \infty \implies J(c\boldsymbol{\theta}) \to 0$$

FAILURE CONDITION — THE MINIMISER DOES NOT EXIST

Under separation the likelihood has no maximum: scaling the weights up always improves it, and the weights run away to infinity. The warning is not a numerical nuisance — it is the solver reporting that the estimate you asked for *does not exist*.

THE REQUIREMENT, CHECKED

Calibration

The difference between a number that like a probability and one that one

What “calibrated” means, precisely

$$P(y = 1 \mid \hat{p}(\mathbf{x}) = q) \;=\; q \qquad \text{for every } q \in (0, 1)$$

Take every passenger the model scored near q. The fraction who actually survived should be about q.

This is a property of a *set* of predictions, not of any one prediction. You cannot check it on a single passenger, which is exactly why it is so easy to ship a model that fails it.

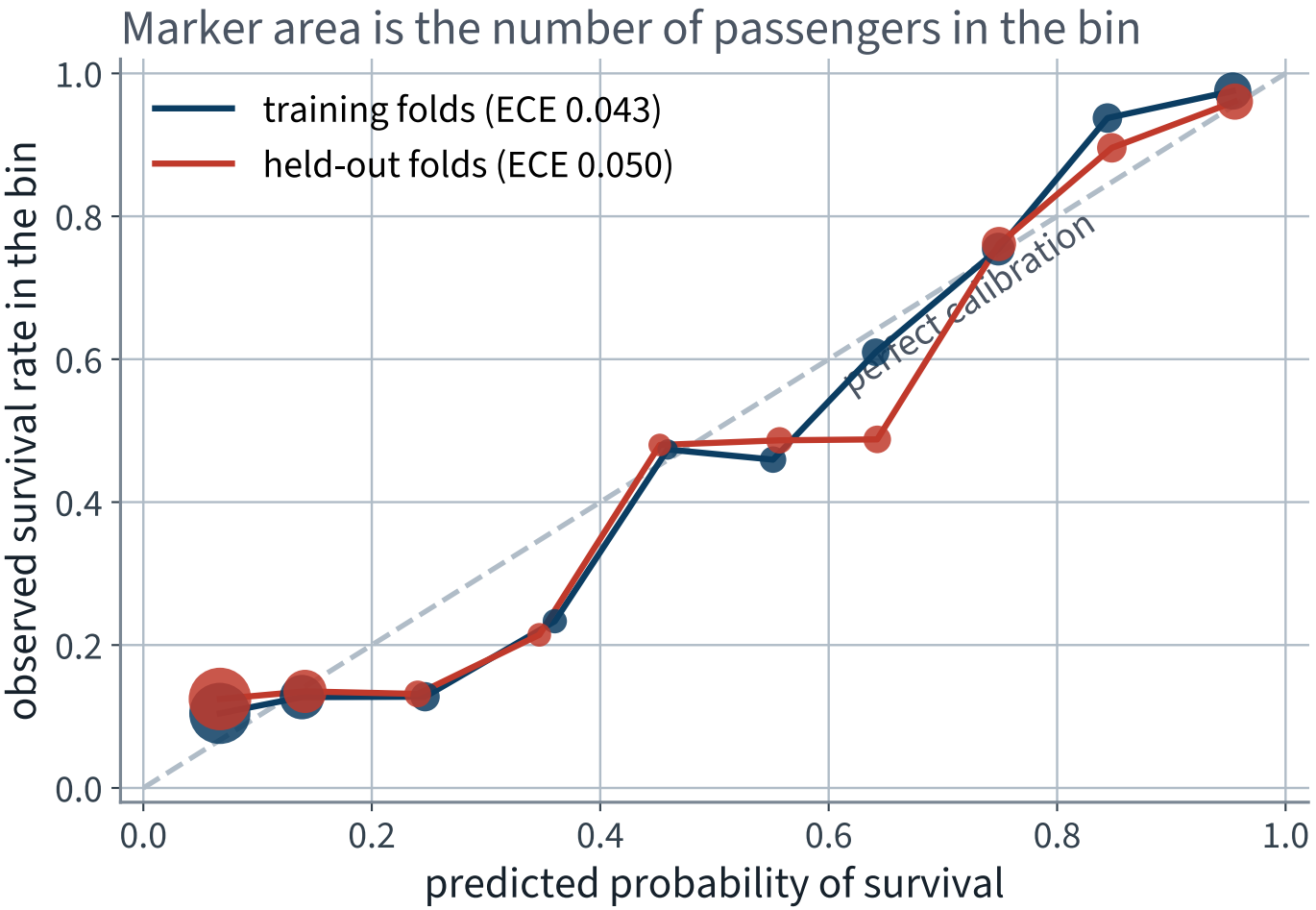
Checking it, out of fold

```
p_oof = cross_val_predict(model, X_train, y_train, cv=cv,
                          method="predict_proba")[:, 1]

edges = np.linspace(0, 1, 11)
idx   = np.clip(np.digitize(p_oof, edges) - 1, 0, 9)
for b in range(10):
    sel = idx == b
    print(f"predicted {p_oof[sel].mean():.3f}   "
          f"observed {y_train.values[sel].mean():.3f}   n={sel.sum()}")
```

`cross_val_predict`, not `predict`: every probability here came from a fit that had never seen that passenger. A reliability diagram drawn on training predictions always looks excellent.

The reliability diagram



Close to the diagonal in the bins that hold most of the passengers — the two ends, where the model is confident.

How to read that diagram without fooling yourself

- Look at the marker **area** first. A bin with six passengers has an observed rate that swings by a sixth on one person
- Deviations in the middle bins are mostly sampling noise; deviations in the crowded end bins are not
- The out-of-fold curve is the one that counts; the training curve is closer to the diagonal by construction

> **WHY IT IS ROUGHLY CALIBRATED AT ALL**
>
> Because log loss is a proper scoring rule and we minimised it directly. The calibration is not a happy accident — it is what the objective was asking for. Expected calibration error, out of fold: **0.050**.

`.predict()` is `.predict_proba() >= 0.5`

FAILURE CONDITION — A BARE LABEL HIDES A DECISION NOBODY MADE

Calling `.predict()` silently applies a cut-off of 0.5, which is cost-optimal only when the two mistakes cost the same. Nothing in the code mentions 0.5. Nothing errors. The accuracy printed is true, and it answers a question the stakeholder did not ask.

Lecture 3 chose an operating point against a stated constraint and defended it. The same discipline applies here, with one addition: **the model produces the probability, and the cut-off is a separate decision taken on top of it**.

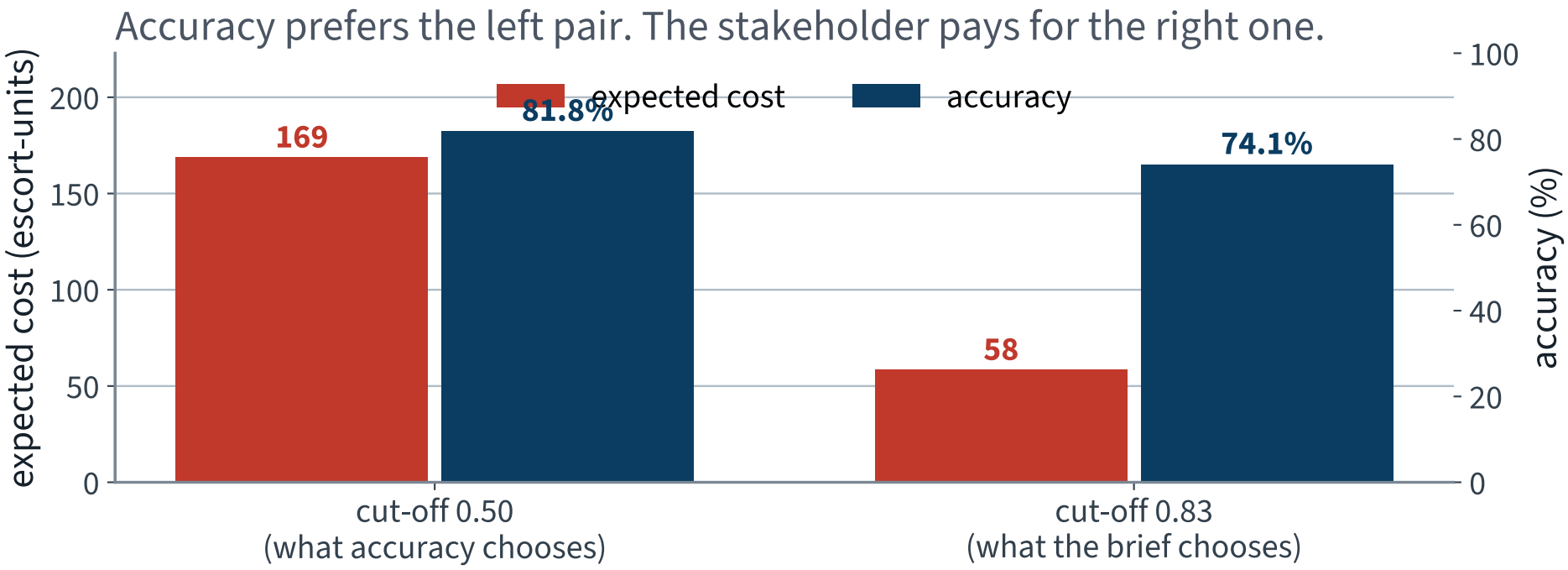
Measure it rather than asserting it

Twenty splits. On each: fit, choose the cut-off on **out-of-fold training predictions** against a stated 10 : 1 cost ratio, then apply both cut-offs to the held-out passengers.

Cut-off	Accuracy	Expected cost
0.50 — the library default	81.8%	✗ 169
0.86 — chosen from the costs, per split	74.1%	59 ✓

✗ The default cut-off costs 110 escort-units more per evacuation, on average — and it is worse on 20 splits out of 20.

The more accurate rule is the more expensive one



✗ Accuracy goes *down* when you do the right thing. If accuracy is the headline you will reject the better rule.

So: report the probability, choose the cut-off separately

- Fit and report on a **proper scoring rule** — log loss, Brier — both cross-validated
- Choose any cut-off **on out-of-fold predictions**, against costs or a capacity that somebody has stated
- Report accuracy at that cut-off as a *consequence*, never as the objective

For a calibrated model the cost-minimising cut-off is $t^* = c_{\mathrm{FN}} / (c_{\mathrm{FN}} + c_{\mathrm{FP}})$ — 0.909 at a 10 : 1 ratio. Our measured optimum sits below it, and that gap *is* the calibration error, expressed in a unit somebody cares about.

FURTHER GROUND

More than two classes

15 minutes · examinable

Softmax regression

One parameter *vector* per class. Score each class, then normalise:

$$s_k(\mathbf{x}) = \boldsymbol{\theta}_k^\top \mathbf{x}, \qquad \hat{p}_k = \frac{\exp\big(s_k(\mathbf{x})\big)}{\sum_{j=1}^{K} \exp\big(s_j(\mathbf{x})\big)}$$

Every $\hat{p}_k > 0$ and they sum to 1 by construction, so the output is a distribution over the K classes without any further work.

Multi*class*, not multi*label*: softmax assigns one class per instance. Four objects in one photograph need K separate binary classifiers, not this.

Its cost: cross-entropy

$$J(\mathbf{\Theta}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)}$$

$y_k^{(i)}$ is 1 if instance i belongs to class k and 0 otherwise, so the inner sum keeps exactly one term: the log of the probability given to the *true* class.

✓ At $K = 2$ this is the log loss, term for term. Logistic regression is the two-class case of softmax, not a different model.

and its gradient is the same shape again

$$\nabla_{\boldsymbol{\theta}_k} J \;=\; \frac{1}{m} \sum_{i=1}^{m} \left(\hat{p}_k^{(i)} - y_k^{(i)} \right) \mathbf{x}^{(i)}$$

Error times feature, for the third time today.

That is not a coincidence. It is a property of this family of losses paired with these output functions, and it is the reason one optimiser fits all of them — including every network in the second half of this course.

What the boundaries look like

Class k is predicted where s_k is the largest score, so the boundary between classes j and k is the set where

$$\left(\boldsymbol{\theta}_k - \boldsymbol{\theta}_j\right)^{\mathsf{T}}\mathbf{x} \;=\; 0$$

A hyperplane. The decision regions are convex polyhedra meeting along flat faces — the multiclass version of the straight line we drew on age and fare.

Which is also the limitation: a class that is not linearly separable from the union of the others cannot be carved out, however many iterations you run.

Which solver, and when

Estimator	What it runs	Use it when
`LogisticRegression`, `lbfgs`	quasi-Newton, batch	the default: the data fits in memory and n is moderate
`LogisticRegression`, `saga`	variance-reduced stochastic	many rows, or an ℓ_1 penalty
`SGDClassifier(loss="log_loss")`	mini-batch descent, explicitly	out-of-core data, or a learning schedule you want to control

All three minimise the same convex cost and, given enough iterations, agree. The choice is engineering: which one gets there fastest on your data. NOT EXAMINABLE

What today's model is still missing

Left open today	What it means	What fixes it
Rank 23 of 29 columns	the minimiser is not **unique**	**✓ a penalty makes it unique**
lbfgs will not converge at degree 4	the minimiser does not **exist**	**✓ a penalty makes it exist**
Held-out log loss quadruples from degree 2 to degree 5	capacity with no counterweight	**✓ a penalty buys most of it back**
We chose the degree by reading the held-out score	a choice with no diagnosis behind it	**✓ learning and validation curves**

The first two are statements about the optimisation problem, not about the passengers.

✓ One idea repairs all four, and it is the next lecture.

The notebook for this lecture

Open Lecture 4 in Colab

- **Runs on free CPU** in about two minutes. The slowest cell is the twenty-split cut-off comparison and it says so
- Gradient descent implemented from the derivation in a dozen lines, then checked against the normal equation to eight decimals — and run at three learning rates, one of which diverges
- Then everything on today's slides on the Titanic manifest: the rank computation, the null-space shift, the coefficients, the calibration bins and the degree sweep

> **WHAT TO CHANGE**
>
> Raise `ETA_DIVERGE` in the learning-rate cell and watch where the cost first becomes `inf`. Then compute $2/\lambda_{\max}$ for that design matrix and check the two agree.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

Summary

- Descent moves along $-\nabla J$; on the MSE that gradient is $\frac{2}{m}\mathbf{X}^{\mathsf{T}}(\mathbf{X}\boldsymbol{\theta} - y)$, and its zero is the normal equation
- The cost is convex, so any stationary point is the global minimum; the step converges iff $\eta < 2/\lambda_{\max}$, and the step *count* is set by the condition number — which is why you scale
- One instance gives an unbiased gradient, which is what makes stochastic and mini-batch descent legitimate rather than merely cheap
- Logistic regression predicts the **log-odds** linearly; coefficients multiply the odds, always relative to a reference level
- Its fit is undefined in two ways: not unique under rank deficiency, and non-existent under separation
- A probability is checkable and a label is not — and `.predict()` is a cut-off nobody chose

References

- Géron, *Hands-On Machine Learning*, 2025 — Chapter 4: gradient descent, polynomial regression, logistic and softmax regression
- Chapter 3 — the precision/recall material this lecture assumes
- Chapter 2 — `ColumnTransformer` and the pipeline discipline, unchanged

The convergence bound $\eta < 2/\lambda_{\max}$ and the contraction factor $(\kappa - 1)/(\kappa + 1)$ go slightly beyond the book's treatment and are examinable as derived here. Convergence rates for non-convex costs are .

BEYOND THE SYLLABUS

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 4

five, in the style of the written paper

Exercises — Lecture 4

1 Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on scaled features. Explain why, in terms of the shape of the cost surface. [5]

2 Your logistic regression on one-hot encoded columns produces coefficients that look wrong, and the design matrix has rank six less than its column count. State the cause and the standard repair. [5]

3 A model reaches 0.80 accuracy and is *badly calibrated*. Explain what that means, and name a decision it would make wrongly that accuracy cannot see. [4]

Solutions on Lecture 5’s deck.

Exercises — Lecture 4 (continued)

4 Why does the log loss have no closed-form minimiser, when squared error does? [4]

5 State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent — and what does not. [4]

Solutions on Lecture 5's deck.

THE FIVE SET AT THE END OF LECTURE 3

Solutions Lecture 3

not part of this lecture

Solutions — Lecture 3

1. On the MNIST 5-detector, a classifier that never fires scores 90.96% accuracy on the training set. State…

✓ The share of the data that is not a 5.

- With no positive prediction, accuracy is exactly the negative class's base rate
- It measures the data, not the model — which is why it is the anchor every other number is read against

2. Prove or disprove: as the decision threshold is lowered, precision increases monotonically.

✓ Disprove — precision is not monotone in the threshold.

- Lowering the threshold admits one instance at a time; recall cannot fall, but precision rises or falls depending on whether that instance is a true or a false positive
- A single false positive admitted at a high-precision point lowers it, and a later true positive can raise it again

Solutions — Lecture 3 (2)

3. A client asks for “90% precision”. Explain why that is not yet a specification, and give the one…

✓ It names no recall, and precision alone is met by predicting almost nothing. Add the recall it must hold at.

- A classifier firing once, correctly, has precision 1.0 and is useless
- An operating point is a pair, and the threshold that achieves it is a decision someone has to own

4. Two models have the same ROC AUC. One is far better on a precision–recall curve. What does that tell…

✓ The positive class is rare.

- ROC uses the false-positive *rate*, whose denominator is the large negative class, so many false positives barely move it
- PR uses precision, whose denominator is what the model predicted, so the same false positives dominate it

Solutions — Lecture 3 (3)

5. The confusion matrix of the never-fires classifier has two zero cells. Name them, and say which single metric…

✓ True positives and false positives are zero; recall exposes it, at 0.0.

- Nothing was predicted positive, so both positive-prediction cells are empty
- Recall divides by the actual positives, so it is 0 however rare they are

NEXT LECTURE

Regularisation, and the bias–variance trade-off

Why the held-out curve turned upward at degree 3, decomposed into three terms — and the one line that repairs both of today's failures